

Optimisation non-linéaire

Dichotomie

Dichotomie. Recherche d'une racine de f continue sur $[a, b]$ ayant une seule racine, par division successive de l'intervalle : $m = \frac{a+b}{2}$.

```

tant que (b - a) > ε :
  m ← (a + b) / 2
  si f(a) · f(m) ≤ 0 : b ← m   sinon : a ← m   retourner m

```

Piège. Minimisation : on cherche le minimum de f , c.-à-d. $f'(x) = 0$ avec $f'(x) < 0$ à gauche et > 0 à droite. On fait donc la dichotomie sur $g(x) = f'(x)$.

Convergence linéaire : 1 bit par itération. Erreur après n itérations :

$$\frac{b-a}{2^n} \leq \varepsilon \Rightarrow n \geq \log_2 \frac{b-a}{\varepsilon}$$

Newton

Newton. Approximation par la tangente : en $f(x_n)$ on prend la tangente, et le zéro de celle-ci est le prochain x_n .

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

$$\text{condition de convergence : } \left| \frac{f(x)f''(x)}{(f'(x))^2} \right| < 1$$

Méthode de la sécante. Approximation de $f'(x_n)$ par taux d'accroissement (pas de dérivée à calculer) :

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}} \Rightarrow x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Piège. Défaillance de Newton : $f'(x_n) \neq 0$, sinon division impossible (tangente horizontale).

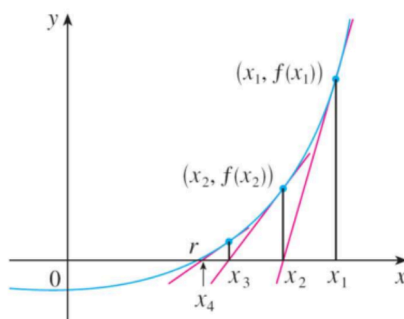


Fig. 1. – Itérations de Newton : zéros successifs des tangentes

Newton à n dimensions

Minimisation d'une fonction $f(x_1, \dots, x_n)$. Matrice hessienne :

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j} \quad \text{en 2D : } H = \begin{pmatrix} f_{x_1 x_1} & f_{x_1 x_2} \\ f_{x_2 x_1} & f_{x_2 x_2} \end{pmatrix}$$

$$x^{i+1} = x^i - (H(x^i))^{-1} \nabla f(x^i)$$

formule d'optimisation

Rappel inversion 2×2 .

$$M = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \Rightarrow M^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

```
Donner x, ε
pour i = 0, 1, 2, ... :
  g ← ∇f(x^(i))
  si ||g|| < ε : retourner x^(i)
  H ← hessienne(x^(i))
  Δ ← H⁻¹ · g   x^(i+1) ← x^(i) - Δ
```

- Convergence quadratique rapide.
- Inverser H coûteux.
- Si f quadratique : minimum exact atteint en 1 itération.

Gradient

Gradient. Depuis x^i on descend dans la direction de $-\nabla f(x^i)$, d'un pas α_i choisi pour minimiser f le long de cette droite.

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \quad x^{i+1} = x^i - \alpha_i \nabla f(x^i)$$

Recherche du pas. Trouver α : poser $h(\alpha) = f(x^i - \alpha \nabla f(x^i))$, puis résoudre $h'(\alpha) = 0$.

```
Donner x, ε
pour i = 0, 1, 2, ... :
  g ← ∇f(x^(i))
  si ||g|| < ε : retourner x^(i)   α_i ← argmin_α f(x^(i) - α·g)   x^(i+1) ← x^(i) - α_i·g
```

- Convergence lente.
- Oscillation vers l'optimal.
- Bonne approximation initiale puis Newton.

Comparaison

Méthode	Vitesse	Robustesse	Point de départ
Dichotomie	lente	robuste	intervalle $[a, b]$
Newton	rapide	peu robuste	proche solution
Gradient	lente	moyenne	arbitraire

Le perceptron

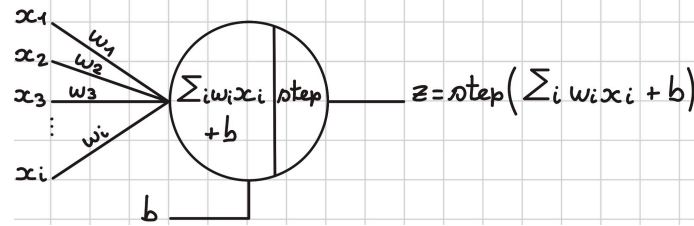


Fig. 2. – Neurone : somme pondérée $\sum_i w_i x_i + b$ suivie d'une fonction d'activation

Perceptron. Combinaison linéaire des entrées suivie d'une fonction d'activation (step) :

$$z = \text{step} \left(\sum_i w_i x_i + b \right)$$

b : biais. t : valeur attendue. o : valeur de sortie.

Fonction de coût.

$$E[\vec{w}] = \frac{1}{2} \sum_l (t_l - o_l)^2$$

Apprentissage — descente de gradient. Mise à jour des poids dans la direction opposée au gradient du coût :

$$w_i \leftarrow w_i - \alpha \frac{\partial E}{\partial w_i}$$

Fonctions d'activation

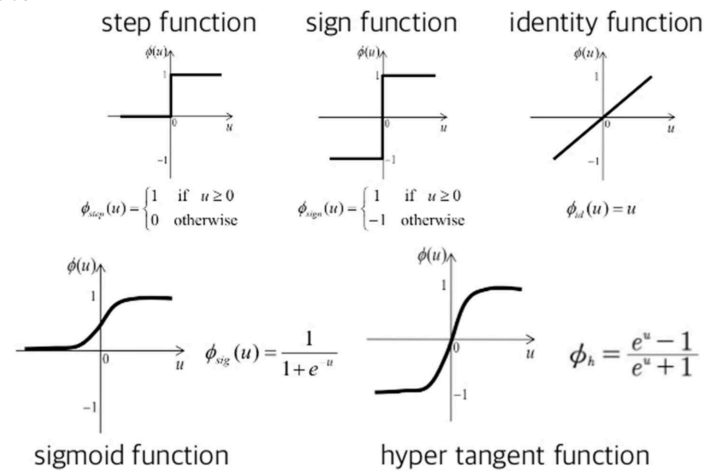


Fig. 3. – Step, sign, identity, sigmoid, hyperbolic tangent

Step : $\varphi(u) = \begin{cases} 1 & \text{si } u \geq 0 \\ 0 & \text{sinon} \end{cases}$
 Sign : $\varphi(u) = \begin{cases} 1 & \text{si } u \geq 0 \\ -1 & \text{sinon} \end{cases}$
 Identity : $\varphi(u) = u$

Sigmoid : $\varphi(u) = \frac{1}{1+e^{-u}}$
 Tanh : $\varphi(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$

Rétropropagation (backpropagation)

Principe. Calcul du gradient $\frac{\partial E}{\partial w}$ pour tous les poids d'un réseau multicouche, par application de la règle de dérivation en chaîne, de la sortie vers l'entrée.

Pour un poids w_{ij} (du neurone i vers j), on décompose :

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \cdot \frac{\partial o_j}{\partial \text{net}_j} \cdot \frac{\partial \text{net}_j}{\partial w_{ij}}$$

$$\text{net}_j = \sum_i w_{ij} o_i, \text{ et } o_j = \varphi(\text{net}_j)$$

Signal d'erreur. On définit $\delta_j = \frac{\partial E}{\partial \text{net}_j}$, propagé de la sortie vers l'entrée :

- couche de sortie : $\delta_j = (o_j - t_j) \varphi'(\text{net}_j)$;
- couche cachée : $\delta_j = \left(\sum_k \delta_k w_{jk} \right) \varphi'(\text{net}_j)$.

$$\frac{\partial E}{\partial w_{ij}} = \delta_j o_i \quad w_{ij} \leftarrow w_{ij} - \alpha \delta_j o_i$$

Piège. δ d'une couche cachée dépend des δ de la couche suivante : il faut donc calculer les δ de la sortie **en premier**, puis remonter. C'est ce qui justifie le terme « rétro »-propagation.

Les activations dérivables (sigmoid, tanh) sont nécessaires : la step n'est pas dérivable, d'où son remplacement pour l'apprentissage par gradient.

Heuristique et métaheuristique

Algorithme du recuit simulé

But. Minimise la fonction coût C sur un ensemble de solutions trop grand pour être énuméré.

Principe. Tient une seule solution à la fois et l'améliore par essais successifs. Avance par petite modification de la solution. Tolère des dégradations pour éviter les minima locaux : grande tolérance au début, puis diminuée.

Formalisme

- 1 **Tirer un voisin**
Tirer une nouvelle solution Y dans le voisinage $\mathcal{N}(x)$ de la solution courante x .
- 2 **Tester**

$$\Delta E = C(Y) - C(x) \quad \begin{cases} > 0 & \text{dégradation} \\ < 0 & \text{amélioration} \end{cases}$$
- 3 **Probabilité d'accepter une dégradation**

$$p = \exp\left(-\Delta \frac{E}{T}\right)$$

$p \approx 0 \rightarrow$ souvent rejeté $p \approx 1 \rightarrow$ souvent accepté. T : tolérance.
- 4 **Baisser la tolérance**

$$T_{n+1} = \alpha \cdot T_n, \quad 0 < \alpha < 1$$

Pseudocode

```

X ← solution de départ
X* ← X      T ← T0

répéter
  Y ← tirer un voisin de X au hasard
  ΔE ← C(Y) - C(X)

  si ΔE ≤ 0 :
    X ← Y
  sinon :
    tirer μ au hasard dans [0,1]
    si μ < exp(-ΔE/T) :
      X ← Y
  si C(X) < C(X*) :
    X* ← X

  T ← α · T
jusqu'à T = 0

retourner X*
```

Cas d'utilisation

- Placement de composants sur circuit / FPGA (minimiser longueur des fils).
- Problème de l'affectation quadratique (QAP).
- Problème des horaires (contraintes ajoutables à tout moment).

Avantages / Inconvénients

Avantages

- Solutions de bonne qualité
- Méthode générale (tout problème dont C est évaluable, voisinage connexe)
- Facile à programmer
- Convergence prouvée vers l'optimum si refroidissement assez lent

Inconvénients

- Temps de calcul : solution exacte exponentielle ; solution approchée à 2 % en $O(N^3)$
- Performances très sensibles au choix de T_0 , α et du voisinage

Piège. En minimisation, $\Delta E > 0$ = dégradation. L'exponentielle ne porte que sur les dégradations ($\Delta E \leq 0$ acceptée d'office). $X^* \neq X$ final : garder le meilleur à part et le renvoyer.

Algorithme de recherche tabou

But. Minimise la fonction coût C sur un ensemble de solutions trop grand pour être énuméré.

Principe. Recherche du minimum en se déplaçant à chaque itération vers le voisin de plus faible coût, même s'il dégrade la fonction coût (sortie d'un minimum local). Possède une mémoire des solutions/mouvements déjà visités pour éviter les retours en arrière. La meilleure solution rencontrée est conservée à part et renvoyée à la fin.

Formalisme

- x : solution courante
- x^* : meilleure solution rencontrée
- C : fonction coût
- $N(x)$: voisinage de x
- T : liste tabou (mouvements interdits)
- $\underline{N}(x)$: voisinage non tabou

Pseudocode

```

x ← solution initiale
x* ← x      T ← liste vide
tant que critère d'arrêt non atteint :
    candidats ← { voisins de x atteints par un mouvement absent de T }
    x ← candidat de plus faible coût      si C(x) < C(x*) :
        x* ← x
    ajouter le mouvement effectué à T
retourner x*
```

Critère d'arrêt

- Solution optimale atteinte (si connue) ou valeur prédéterminée
- Nombre d'itérations (ou temps de calcul) ou nombre d'itérations sans amélioration de x^*

Piège. « Plus faible coût » = minimum sur les voisins, pas forcément inférieur à $C(x)$. Dans un minimum local tous les voisins sont pires : le coût remonte, ce qui fait sortir du minimum. La mémoire stocke les mouvements. FIFO de longueur fixe t .

Affectation quadratique (QAP)

Définition. Répartition de n objets dans un espace de n emplacements de sorte à minimiser le coût total des déplacements entre eux.

Données

- n objets ; $F(i, j)$: flot (poids de connexion) entre les objets i et j
- n emplacements ; $D(r, s)$: distance entre les emplacements r et s

Codage

Une solution est une permutation π , où $\pi(i)$ donne l'emplacement de l'objet i . Espace des solutions : $n!$ permutations.

Coût d'un placement

$$C(\pi) = \sum_{i,j=1}^n F(i,j) \cdot D(\pi(i), \pi(j))$$

pour chaque paire d'objets : flot $\times$ distance entre leurs emplacements, puis somme.

Exemple

Avec $\pi = [1, 3, 4, 5, 2]$ et les matrices ci-dessous :

Flots F

$$\begin{pmatrix} 0 & 5 & 2 & 4 & 1 \\ 5 & 0 & 3 & 0 & 2 \\ 2 & 3 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 5 \\ 1 & 2 & 0 & 5 & 0 \end{pmatrix}$$

Distances D

$$\begin{pmatrix} 0 & 1 & 1 & 2 & 3 \\ 1 & 0 & 2 & 1 & 2 \\ 1 & 2 & 0 & 1 & 2 \\ 2 & 1 & 1 & 0 & 1 \\ 3 & 2 & 2 & 1 & 0 \end{pmatrix}$$

Somme par objet i (un terme par paire, $F(i, j) \cdot D(\pi(i), \pi(j))$) : $22 + 12 + 7 + 22 + 15 = 78$.

Problèmes à satisfaction de contraintes (PSC)

But. Modéliser un problème de décision sous forme générique (variables, domaines, contraintes) pour le résoudre avec des algorithmes universels, indépendants de la signification du problème.

Principe. Affecter une valeur à chaque variable, prise dans son domaine, sans violer aucune contrainte. Une affectation complète qui respecte toutes les contraintes est une **solution**.

Formalisme

- $X_1, \dots, X_n$: variables (inconnues)
- D_i : domaine de X_i (valeurs possibles, fini)
- A : affectation courante $\{X_1 \leftarrow v_1, \dots, X_k \leftarrow v_k\}$
- A **valide** : aucune contrainte violée par les variables affectées de A .
- $C_1, \dots, C_p$: contraintes (combinaisons de valeurs permises)
- k : nb de variables affectées ($0 \leq k \leq n$)
- d : taille des domaines
- **PSC binaire** : toute C_k lie au plus 2 variables $\rightarrow$ **graphe de contraintes** (nœud = variable, arête = contrainte).

Backtracking

Définition. Recherche en profondeur : on affecte les variables une par une. Si l'on ne peut plus affecter de valeur valide (en fonction de la contrainte) à une variable, on **revient en arrière** sur la précédente pour essayer sa valeur suivante.

Forward checking

Définition.

Heuristiques

- **Variable la plus contrainte** : affecter d'abord celle au plus petit domaine restant (minimise le facteur de branchement).
- **Variable la plus contraignante** : à égalité, celle liée au plus de contraintes sur des variables non affectées.
- **Valeur la moins contraignante** : essayer d'abord la valeur qui retire le moins d'options aux variables voisines.

Complexité

- $O(d^n)$ affectations complètes : recherche exhaustive irréaliste.
- Backtracking + forward checking + heuristiques rendent la recherche praticable.

Algorithmes génétiques

But. Recherche d'une bonne solution à un problème en faisant évoluer une population de solutions candidates sur plusieurs générations.

Principe. On part d'un ensemble de solutions tirées au hasard. À chaque itération, les individus les mieux notés sont conservés et combinés entre eux (croisement), avec quelques modifications aléatoires (mutation), pour former la génération suivante. La population s'améliore ainsi jusqu'à obtenir une solution satisfaisante. La meilleure solution rencontrée est renvoyée à la fin.

Bases biologiques

- **Hérédité** : les descendants sont similaires aux parents
- **Sélection** : les mieux adaptés survivent

- **Variabilité** : des changements aléatoires perturbent l'hérédité et augmentent la diversité

Terminologie

- **Individu** : une solution candidate
- **Gène** : une partie de la solution
- **Génotype** : le codage de la solution
- **Phénotype** : la solution réelle codée par le génotype

- **Population** : groupe de solutions testées en même temps
- **Fitness** : fonction qui note la qualité d'un individu

Pseudocode

```
Population : ensemble de solutions candidates
Individu : une solution candidate
Fitness : fonction qui note la qualité d'un individu
Parents : individus retenus pour se reproduire
Descendants : nouveaux individus issus des parents
```

```
Population ← solutions tirées au hasard
noter chaque Individu avec Fitness
```

```
répéter jusqu'à solution satisfaisante :
```

```
  Parents ← les individus les mieux notés
  Descendants ← crossover des Parents
  Descendants ← mutation des Descendants
  noter chaque Descendant avec Fitness
  Population ← Descendants
```

```
renvoyer le meilleur Individu
```

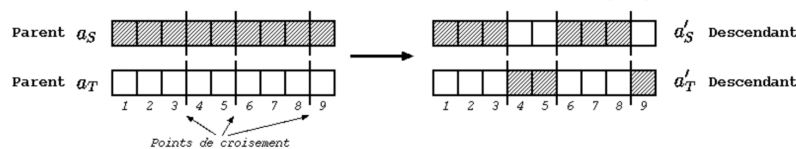
Opérateurs d'évolution

Croisement (crossover) : combine deux parents en échangeant des segments de leur génotype. Opérateur d'*exploitation* : recombine des caractères déjà présents dans la population.

Mutation : modifie aléatoirement un gène (probabilité p_m par gène). Opérateur d'*exploration* : indispensable pour éviter les optimums locaux.

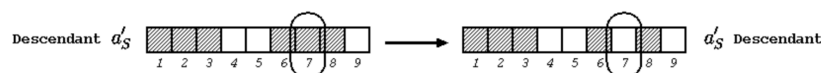
Croisement / recombinaison (crossover)

- choix des individus formant la nouvelle population



Mutation

- modification aléatoire d'un individu



Sélection

Proportionnelle. Probabilité de sélection proportionnelle à la fitness : $p_i = f_i / \sum_j f_j$ (roulette biaisée). Les meilleurs ont plus de chance, les moins bons en conservent une (leurs gènes peuvent être utiles).

Remplacement

- **Générationnel** : toute la population est remplacée par les descendants
- **Élitisme** : conservation systématique du (des) meilleur(s) individu(s) → fitness du meilleur non décroissante

Critère d'arrêt

- Optimum atteint (si connu) ou valeur prédéterminée
- Limite de temps / nombre de générations
- Aucune amélioration pendant plusieurs générations

Piège. Sélection $\neq$ remplacement : la sélection choisit *qui se reproduit*, le remplacement choisit *qui constitue la génération suivante*. L'élitisme protège le meilleur mais réduit la diversité $\rightarrow$ risque de convergence prématurée vers un optimum local. Point clef : maintenir la diversité génétique, car sa perte est irréversible en pratique.

Avantages et inconvénients

Avantages

- Bon rapport coût / résultat sur une grande classe de problèmes
- Parallélisme intrinsèque
- Robuste, tolérant aux fautes
- Applicable sans connaissance préalable du domaine
- Simple à programmer

Inconvénients

- Pas de garantie de convergence en temps fini
- Fondements théoriques faibles
- Gourmands en calcul (mais parallélisables)
- Chaque individu doit être évalué, même les inadaptés

Particulièrement adaptés lorsque le problème comporte beaucoup de paramètres, des paramètres interdépendants, et des optimaux locaux. « *Everything is problem dependent* » : pas une panacée universelle.

Programmation génétique

But. Utiliser l'évolution pour construire un **programme**, et non plus un simple jeu de paramètres. Proposée par Koza (1989). Aspect quasi-universel.

Principe. L'individu n'est plus un chromosome de longueur fixe mais un **arbre de taille variable** représentant une expression ou un programme. Les nœuds internes sont des **fonctions** (+, $\times$, sin, if-then-else, while...) et les feuilles des **valeurs terminales** (variables x , y , constantes, vrai, faux). On cherche donc la structure elle-même, pas seulement ses paramètres.

Ensembles fonctions / terminaux

Clos : toute fonction doit accepter en argument tout terminal ou tout type retourné par les fonctions.

Suffisant : il doit exister au moins une solution constructible à partir de ces ensembles.

Opérateurs

- **Croisement** : échange de deux sous-arbres entre deux parents
- **Mutation** : remplacement d'un nœud ou d'un sous-arbre

Exemples

- **Régression symbolique** : trouver la formule $f(x)$ ajustant un nuage de points (structure inconnue $\rightarrow$ arbre)
- **Vie artificielle** : créatures virtuelles de Karl Sims, Golem Project

Piège. AG classique = chromosome de longueur **fixe** (on cherche des valeurs dans une structure connue). PG = arbre de taille **variable** (on cherche la structure). Contrepartie : croissance incontrôlée de la taille des arbres au fil des générations.

Algorithme de Fourmis

1. Historique & Origine

Origine. Proposé par **Marco Dorigo et al.** dans les années **1990** pour la recherche de chemins optimaux dans un graphe. S'inspire du comportement des fourmis cherchant un chemin entre leur colonie et une source de nourriture.

Individuellement, les fourmis ont des capacités cognitives **limitées**, mais collectivement elles sont capables de trouver le **chemin le plus court** entre une source de nourriture et leur nid.

2. Mécanisme des phéromones

Phéromone. Substance odorante déposée par une fourmi sur son passage, détectable par les autres fourmis. Les phéromones sont **volatiles** : elles s'évaporent avec le temps.

- 1 **Dépôt**
Une fourmi (« éclaïreuse ») explore l'environnement au hasard. Si elle découvre une source de nourriture, elle rentre au nid en laissant une **piste de phéromones**.
- 2 **Attraction**
Les fourmis passant à proximité ont tendance à **suivre** la piste phéromonnée et la **renforcent** en revenant au nid.
- 3 **Sélection du plus court**
Si deux pistes existent, la plus **courte** est parcourue par **plus de fourmis** par unité de temps → elle accumule plus de phéromones → elle devient plus attractive.
- 4 **Disparition de la longue piste**
La piste longue reçoit moins de renforcement et **disparaît** par évaporation. À terme, l'ensemble de la colonie converge vers le chemin le plus court.

3. Description générale de la méthode

Méta-heuristique à population. Les algorithmes de colonies de fourmis sont des **algorithmes multi-agents probabilistes**. Chaque solution est représentée par une fourmi se déplaçant dans l'espace de recherche. Les fourmis marquent les **meilleures solutions** et tiennent compte des marquages précédents pour orienter leur recherche.

Les 4 éléments de base d'un algorithme de fourmis :

1. **Représentation** appropriée du problème sous forme de graphe.
2. **Attractivité heuristique** des arêtes du graphe sous-jacent (η).
3. **Règle de mise à jour** des phéromones sur les arêtes (σ).
4. **Règle de transition probabiliste** fonction des phéromones et de l'attractivité heuristique.

Dans les versions adaptées aux **problèmes combinatoires**, la solution est construite de manière **itérative**.

4. Application au voyageur de commerce (TSP)

Règles de déplacement d'une fourmi

À chaque étape, la fourmi choisit la prochaine ville selon 5 règles :

1. Elle ne peut **visiter qu'une fois** chaque ville.
2. Plus une ville est **loin**, moins elle a de chances d'être choisie (**visibilité**).
3. Plus la piste de **phéromone** sur un arc est intense, plus il est probable qu'il soit choisi.
4. Une fois son trajet terminé, la fourmi dépose **plus de phéromones** si le trajet est **court**.
5. Les pistes de phéromones **s'évaporent** à chaque itération.

Probabilité de transition

Probabilité d'aller du sommet i au sommet j pour la fourmi k à l'itération t :

$$p_{ij}^k(t) = \begin{cases} \frac{\sigma_{ij}^\alpha(t) \eta_{ij}^\beta}{\sum_{l \in J_i^k} \sigma_{il}^\alpha(t) \eta_{il}^\beta} & \text{si } j \in J_i^k \\ 0 & \text{sinon} \end{cases}$$

J_i^k = sommets atteignables · $\sigma_{ij}(t)$ = phéromone sur l'arc (i, j) · $\eta_{ij} = 1/d_{ij}$ (visibilité) · α, β = paramètres d'importance

Quantité de phéromone déposée

$$\Delta\sigma_{ij}^k(t) = \begin{cases} \frac{Q}{L^k} & \text{si } (i, j) \in T^k(t) \\ 0 & \text{sinon} \end{cases}$$

$T^k(t)$ = arcs parcourus par la fourmi k · $L^k(t)$ = longueur du parcours · Q = paramètre de réglage

Plus le parcours est **court**, plus la quantité déposée est **importante**.

Mise à jour des phéromones

Sur chaque arc, on applique l'évaporation puis on ajoute les traces de toutes les fourmis :

$$\sigma_{ij}(t+1) = (1-\rho)\sigma_{ij}(t) + \sum_{k=1}^m \Delta\sigma_{ij}^k(t)$$

m = nombre total de fourmis · $\rho \in]0, 1[$ = taux d'évaporation

Piège. Si ρ est trop grand, les phéromones disparaissent trop vite → pas de mémoire collective. Si ρ est trop petit, l'algorithme converge prématurément vers une solution sous-optimale.

Pseudo-code général

Initialiser les traces de phéromones $\sigma_{ij} \leftarrow \sigma_0$ (petites valeurs)

Répéter

Sélectionner aléatoirement K fourmis sur M villes

Pour i de 1 à M faire

Pour j de 1 à K faire

 Choisir la ville suivante via la règle probabiliste $p_{ij}^k(t)$

Fin pour

Fin pour

Mettre à jour les phéromones : $\sigma_{ij}(t+1) \leftarrow (1-\rho)\sigma_{ij}(t) + \sum_k \Delta\sigma_{ij}^k(t)$

Jusqu'à condition de fin (nb. d'itérations ou convergence)

5. Exemple TSP — 5 sommets

Paramètres : 4 fourmis, $\alpha = \beta = 1$, $Q = 1$, $\rho = 1/2$, 8 itérations, phéromone initiale = 1 sur chaque arc, toutes les fourmis partent du sommet 1.

Matrice des distances :

d	1	2	3	4	5
1	x	1	2	2	2
2	1	x	1	2	2
3	2	1	x	1	2
4	2	2	1	x	10
5	2	2	2	10	x

Convergence observée :

- Itér. 1–3 :** Les fourmis explorent des chemins variés (longueurs entre 8 et 17).
- Itér. 4–6 :** Convergence progressive vers les meilleurs chemins (longueur 8–9).
- Itér. 7–8 :** Toutes les fourmis empruntent le tour optimal.

Tour optimal. 1 → 2 → 5 → 3 → 4 → 1
Longueur = 8

6. Applications & Conclusion

Domaines d'application

Les algorithmes de colonies de fourmis ont été appliqués à de nombreux **problèmes d'optimisation combinatoire** :

- Affectation quadratique** — ordonnancement, placement de circuits.
- Repli de protéines** — bioinformatique.
- Routage de véhicules** — logistique.
- Routage réseau** — adaptation dynamique si le graphe change en cours d'exécution.

Conclusion

Résumé de la méthode. Un algorithme de fourmis alterne, pendant un certain nombre d'itérations, deux procédures :

- **Construction/modification** de solutions par les fourmis (règle probabiliste).
- **Mise à jour des phéromones** sur les arêtes du graphe (évaporation + dépôt).

La **phéromone** encode la performance des solutions construites.

L'**heuristique locale** encode l'attractivité intrinsèque des arcs.

Réseaux et graphs

Introduction

Grphe. $G = (V, E)$: V ensemble des sommets, E ensemble des arêtes. Une arête est un couple (u, v) .

Complexité : temps / mémoire, en fonction de $|V|$ et $|E|$.

Non orienté. Paire non ordonnée : $(u, v) = (v, u)$.

Orienté (digraphe). Paire ordonnée : $(u, v) \neq (v, u)$.

Adjacence. u adjacent à v si $(u, v) \in E$. Non orienté : dans les deux sens. Orienté : pas l'inverse.

Pondéré. Un poids associé à chaque arête : $w : E \rightarrow \mathbb{R}$.

Chaînes et cycles

Chaîne. Suite de k arêtes entre sommets.

Chaîne élémentaire. Chaîne dont tous les sommets sont distincts, sauf les extrémités.

Boucle. Arête (u, u) : chaîne élémentaire de longueur 1.

Cycle. Chaîne de longueur ≥ 3 dont les deux extrémités sont identiques. Élémentaire si la chaîne l'est.

Chemins et circuits. Équivalents de chaîne et cycle pour un digraphe.

Graphe simple : digraphe sans boucle.

Connexité et arbres

Connexité. Un chemin relie n'importe quelle paire de sommets.

Forte connexité. Digraphe : tout est relié, mais dans les deux sens.

Arbre non orienté. Acyclique et connexe.

Arbre enraciné. Acyclique, chemin unique vers la racine.

Structures de données

Densité : $|E| \approx |V|^2$ dense ; $|E| \approx |V|$ peu dense.

Matrice d'adjacence. $A(n \times n)$, $A(i, j) = 1$ si sommets connectés. Symétrique si non orienté. Espace : $O(|V|^2)$.

Liste d'adjacence. Pour chaque sommet, la liste de ses voisins. Espace : $O(|V| + |E|)$.

Parcours de graphes

Principe. Depuis une source, on traite les sommets un par un. Traiter un sommet = le marquer visité, puis insérer dans la structure ses **sommets adjacents** (voisins) non visités. L'**ordre d'extraction** fixe l'ordre de parcours.

- **Largeur (BFS) : FIFO**, extraction dans l'ordre d'insertion.
- **Profondeur (DFS) : LIFO**, extraction dans l'ordre inverse.

Complexité $O(|V| + |E|)$.

Exemple

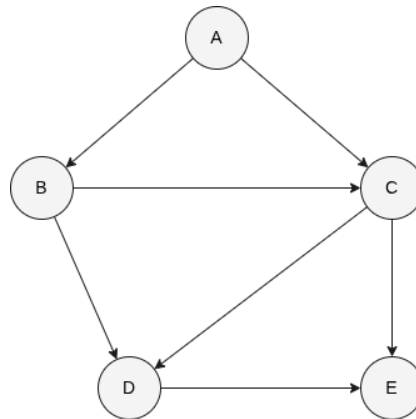


Fig. 5. – Voisins dans l'ordre de la liste d'adjacence

Largeur — FIFO

Extraction par la gauche, insertion à droite.

```

FIFO[A] visite A
FIFO[B C] visite B
FIFO[C D] visite C
FIFO[D E] visite D
FIFO[E] visite E
  
```

Ordre : **A B C D E**

Profondeur — LIFO

Extraction et insertion du même côté.

```

LIFO[A] visite A
LIFO[B C] visite C
LIFO[B E D] visite D
LIFO[B E E] visite E
LIFO[B] visite B
  
```

Ordre : **A C D E B**

Plus court chemin — Dijkstra

Principe. Plus courts chemins depuis une source s dans un graphe pondéré à **poids positifs**. On maintient pour chaque sommet v :

- $\text{cost}[v]$: coût provisoire du plus court chemin connu (0 pour s , ∞ sinon) ;
- $\text{pred}[v]$: prédécesseur sur ce chemin.

Tant qu'il reste des sommets non visités : extraire celui de cost minimal, le marquer définitif, puis relâcher ses arcs sortants.

$$\text{cost}[u] + w(u, v) < \text{cost}[v] \Rightarrow \begin{cases} \text{cost}[v] \leftarrow \text{cost}[u] + w(u, v) \\ \text{pred}[v] \leftarrow u \end{cases}$$

relaxation de l'arc (u, v)

Piège. Poids ≥ 0 obligatoire : un sommet extrait est figé définitivement. Un arc négatif pourrait l'améliorer après coup $\rightarrow$ résultat faux.

Complexité avec tas binaire : $O((|V| + |E|) \log |V|)$.

Exemple — $\text{source} = 1$

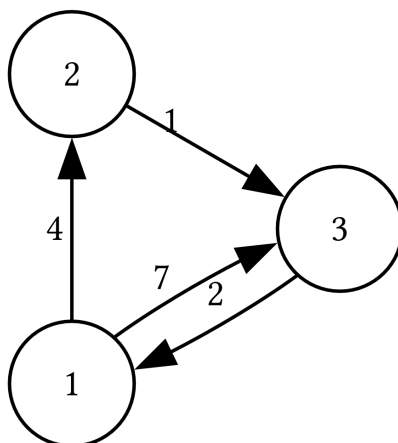


Fig. 6. — Graphe pondéré orienté

Étape 0 — initialisation.
 $\text{cost}[1] = 0$, les autres à ∞ .

Sommet	Coût	Préd.
1	0	—
2	∞	—
3	∞	—

Étape 2 — extraction de **2** (coût min 4), figé.

Relaxation de $(2, 3)$: $4 + 1 = 5 < 7 \rightarrow$ on améliore 3 et son prédécesseur devient 2.

Sommet	Coût	Préd.
1	0	—
2	4	1
3	5	2

Chemins obtenus : 1 ; 1 $\rightarrow$ 2 (coût 4) ; 1 $\rightarrow$ 2 $\rightarrow$ 3 (coût 5).

Étape 1 — extraction de **1** (coût min 0), figé.

Relaxation des arcs sortants : $(1, 2)$ donne $0 + 4$, $(1, 3)$ donne $0 + 7$.

Sommet	Coût	Préd.
1	0	—
2	4	1
3	7	1

Étape 3 — extraction de **3** (coût 5), figé.

Son seul arc $(3, 1)$ pointe vers un sommet déjà définitif : aucune relaxation. Terminé.

Sommet	Coût	Préd.
1	0	—
2	4	1
3	5	2

Plus courts chemins — Floyd-Warshall

Principe. Plus courts chemins entre **toutes les paires** de sommets. On autorise progressivement les sommets $1, \dots, n$ comme **intermédiaires**. À l'étape k , $D^k(i, j)$ est le coût minimal de i à j n'empruntant que des intermédiaires dans $\{1, \dots, k\}$.

- D^0 : matrice d'adjacence pondérée (0 sur la diagonale, ∞ si pas d'arc).
- $P^k(i, j)$: prédécesseur de j sur ce chemin.

$$D^k(i, j) = \min(D^{k-1}(i, j), D^{k-1}(i, k) + D^{k-1}(k, j))$$

passer par k améliore-t-il le chemin $i \rightarrow j$?

Si le chemin via k est meilleur : $P^k(i, j) \leftarrow P^{k-1}(k, j)$. Admet les poids négatifs (pas de cycle négatif). Complexité $O(|V|^3)$.

Exemple

$k = 0$ — initialisation (matrice d'adjacence) :

D^0

P^0

	1	2	3
1	0	4	7
2	∞	0	1
3	2	∞	0

	1	2	3
1	–	1	1
2	–	–	2
3	3	–	–

$k = 1$ — intermédiaire 1. Seul gain : $D(3, 2) = D(3, 1) + D(1, 2) = 2 + 4 = 6 < \infty$.

D^1

P^1

	1	2	3
1	0	4	7
2	∞	0	1
3	2	6	0

	1	2	3
1	–	1	1
2	–	–	2
3	3	1	–

$k = 2$ — intermédiaire 2. Gain : $D(1, 3) = D(1, 2) + D(2, 3) = 4 + 1 = 5 < 7$, préd. $\leftarrow 2$.

D^2

P^2

	1	2	3
1	0	4	5
2	∞	0	1
3	2	6	0

	1	2	3
1	–	1	2
2	–	–	2
3	3	1	–

$k = 3$ — intermédiaire 3. Gain : $D(2, 1) = D(2, 3) + D(3, 1) = 1 + 2 = 3 < \infty$, préd. $\leftarrow 3$.

D^3

P^3

	1	2	3
1	0	4	5
2	3	0	1
3	2	6	0

	1	2	3
1	–	1	2
2	3	–	2
3	3	1	–

Flot maximal — Ford-Fulkerson

But. Réseau orienté avec une source s , un puits t , et un plafond (capacité) sur chaque arête. On cherche la plus grande quantité de flot envoyable de s à t sans dépasser le plafond d'aucune arête. Le flot se répartit sur plusieurs chemins en parallèle.

Flot. Un nombre f posé sur chaque arête = quantité qui y passe.

- $0 \leq f \leq w$ (plafond) ;
- sur tout sommet sauf s et t : flot entrant = flot sortant.

Valeur $|f|$ = flot sortant de s .

Réseau résiduel. Pour chaque arête $u \rightarrow v$ portant f sur un plafond w :

- arête **avant** $u \rightarrow v$ valant $w - f$ (place libre) ;
- arête **arrière** $v \rightarrow u$ valant f (flot annulable).

On garde les arêtes de valeur > 0 . C'est dans ce graphe qu'on cherche les chemins.

Chemin améliorant. Chemin de s à t dans le réseau résiduel. On y pousse $\delta = \min$ des valeurs du chemin.

Algorithme.

- 1 Initialisation**
Flot = 0 sur toutes les arêtes.
- 2 Réseau résiduel**
Le construire à partir de l'état actuel du flot.
- 3 Chemin améliorant**
Chercher un chemin $s \rightarrow t$ dedans. Aucun $\rightarrow$ fin.
- 4 Goulot**
 δ = plus petite valeur du chemin.
- 5 Mise à jour**
Arête avant $\rightarrow f + \delta$; arête arrière $\rightarrow f - \delta$.
- 6 Boucle**
Retour à l'étape 2.

Piège. L'arête **arrière** permet d'annuler un flot mal placé pour le réacheminer. On ne la choisit pas exprès : si un chemin du résiduel l'emprunte, l'annulation ($-\delta$) se fait à la mise à jour. Inutile dans beaucoup de cas.

Maximalité — coupe minimale. Une coupe (S, T) partage les sommets avec $s \in S, t \in T$. Sa capacité $C(S, T)$ = somme des plafonds des arêtes de S vers T .

$$|f| = C(S, T)$$

$$\text{flot max} = \text{coupe min}$$

Le flot est maximal quand plus aucun chemin $s \rightarrow t$ n'existe dans le résiduel. À ce moment S = sommets atteignables depuis s dans le résiduel final, et la coupe (S, T) est saturée.

Complexité (Edmonds-Karp, chemin par BFS) : $O(|V| \cdot |E|^2)$.

Arbre couvrant minimal (MST)

But. Dans un graphe non orienté, pondéré et connexe, sélectionner un sous-ensemble d'arêtes qui relie tous les sommets, sans cycle (un arbre), de poids total minimal. Pour n sommets : exactement $n - 1$ arêtes.

Prim. Fait grossir un seul arbre depuis un sommet de départ. À chaque étape, ajoute l'arête de poids minimal reliant un sommet de l'arbre à un sommet hors de l'arbre.

Kruskal. Trie toutes les arêtes par poids croissant. Les parcourt dans l'ordre et ajoute chaque arête sauf si elle crée un cycle (extrémités déjà reliées). Détection de cycle par union-find.

Les deux sont gloutons et donnent un MST de même poids total.

Exemple

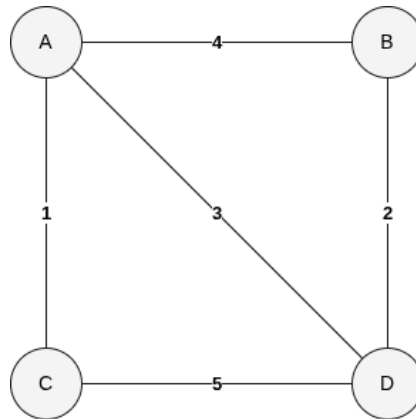


Fig. 7. – Graphe non orienté pondéré

Prim depuis A

Étape 0 — init. $A = 0$.

Sommet	Coût	Préd.
A	0	–
B	∞	–
C	∞	–
D	∞	–

Étape 2 — intégrer C (coût min 1). $C \rightarrow D = 5 > 3$: pas de maj.

Sommet	Coût	Préd.
A	0	–
B	4	A
C	2	A
D	3	A

Étape 4 — intégrer B (2). Arbre complet : $A - C, A - D, B - D$, poids = 6.

L'étape 3 est le point clé : B valait 4 (via A), un chemin moins cher est découvert (via D), on relâche son coût à 2.

Kruskal

Arêtes triées : $AC : 1, BD : 2, AD : 3, AB : 4, CD : 5$.

Arête	Sûre	Forêt F	A (sûres)
$AC : 1$	oui	$\{A, C\} \{B\} \{D\}$	A-C
$BD : 2$	oui	$\{A, C\} \{B, D\}$	A-C, B-D
$AD : 3$	oui	$\{A, B, C, D\}$	A-C, B-D, A-D
$AB : 4$	non	$\{A, B, C, D\}$	– (cycle)

L'arête AB est rejetée : A et B sont déjà dans le même groupe, l'ajouter créerait un cycle. Arrêt à $n - 1 = 3$ arêtes, poids = 6.

Étape 1 — intégrer A. Maj des voisins.

Sommet	Coût	Préd.
A	0	–
B	4	A
C	1	A
D	3	A

Étape 3 — intégrer D (3). $B \rightarrow D = 2 < 4$: B mis à jour.

Sommet	Coût	Préd.
A	0	–
B	2	D
C	2	A
D	3	A